EE 285

PROJECT REPORT

Group 1

Members: Connor Crimmins, Flatrim Dreshaj, Greta Nelson

Report by: Greta Nelson

SUBMISSION DATE: 10 April 2024

Project Overview

For this project, we wanted to create a mock power grid in which the user clicks different conditions to divert power to different houses and computes how much each house owes the power company. We split this project into different modules and used a modular style development process. We split this project into three separate parts; user interface, money/profit computation, and power path in the grid.

Related work

Power Grid

For this part, Connor did research on how to get power from the power plant to the houses. He used the help of a senior Power engineering student here at Iowa State. He found that it does take a lot of steps to get it to go to the houses. First generation, then it goes to the power lines, then to substations, then to utility lines, and finally to the houses. The reason for including this part, well it is the main idea of the code; Power engineering. It was difficult to make this possible since all the houses are different, but we got it to work, you will see this more, in Flatrim's work of the code.

Money and Profit

For this part, Greta did research on how much it costs to produce power and how much is typically charged per kilowatt-hour used. She found that it costs around $0.09 to produce one kilowatt-hour and usually companies charge around $0.15 for each kilowatt-hour used. Power production costs were based on a solar field approach and the average charged per kilowatt-hour was found as an average of the whole United States region. The reason for including this part is to see if the problem is feasible. We wanted to make sure that a company could profit from this power production and supply. This problem is more of an administrative problem, but it is good to know the finances of the company and is a necessary process.

User Interface

For this part, Flatrim did research into C libraries that allowed for GUI style development OOP libraries. The specific one he chose was the GTK library GTK 4. The user interface was created using this OOP style programming by creating widgets, specifically: Window, buttons, grid. By combining these three widgets we can set buttons to be locked in the grid and introduce call back functions to the buttons when clicked. Using these tools the main logic that Flatrim used for the UI was User Choice-> Callback function -> initialize new buttons ->User Choice, this kind of repeating system where, based on the user choices, specific variables are changed which control which type of buttons are initialized and how that branch out into a decision tree. Using this system Flatrim combined the grid and profit logic of the code that Greta and Connor wrote and implemented that logic with the tools of the GTK library. With everything combined the flow of the program goes like so: First the user clicks to initialize the grid, then the user can click the number 3 to initialize 3 houses, after initializing the 3 houses user needs to click the start power generation button which starts the process of generating power. From this point the user has three options for transmission power. High power, medium power, low power. All three of these options send the user into a decision tree where ex. If user chooses high power, they need to subsequently choose to send that high power through the high voltage power line and step down the voltage 3 times. If the user fails to go through this decision tree by parameters which are deemed unsafe by the program, the failsafe will block the action of sending power to a house and the user will

need to click the Reset Grid button which is always available to the user, acting as a “Back” button. If the user chooses the safe parameters, the user is prompted to choose a house to direct that power to, after directing the power the user can monitor the power draw, the power sent to the house and how much money does the user make to the electric company. After showing this data, the user can reset the grid and start over again or control/monitor other houses by directing power to other houses.

Project Technical Description

For this project, we used a modular style development. Connor and Greta created their pieces and Flatrim implemented the code to conform to the GTK style user interface structure.

Connor developed the path of the power system by initializing power and creating different operations to perform different functions such as directing the power. He used a function approach of Boolean operators along with different loops and branches to create these functions. When making the code for this section, it faced a lot of challenges. In the beginning stages of the code, Connor was trying to get it to just create power. At that time, Connor was able to do it swiftly, however Connor did have some other thoughts in his head that he wanted to add to the code but did not. At first, he was going to implement weather into the code, so it would change it up a bit. Connor was going to add, perfect weather (code runs perfectly), a bad storm (utility poll gets knocked down, so no power could reach the houses), and a snowstorm (car crashes into utility poll, therefore the power cannot reach the houses. Connor wanted to add this in, so that it would give the code some more uniqueness to it however, when he talked to the EE senior, he told him that weather does not really affect these types of things, so he abandoned the idea. After that he worked on trying to get the power to now go to high voltage power lines. In this section of the code, Connor ran into a few issues. Whenever he would type in where he wanted the power to go, it would output that the entered characters are incorrect, and you must type ‘hvpl.’ This would then loop the same thing three times, and finally ask the question again. He got confused because he wrote what he was supposed to. After some debugging, he found out that he messed up with my spacing, and that he used the wrong type of loop. After some time, the code worked to that extent. His next step was to get the power to go from the high voltage power lines, to substations, that would lower the voltage. This part of the code faced a lot of problems. This was because when Connor tried to run the code, it would loop three times if the user selected high power in the beginning, 2 times if it were medium, and once if it were only once. The problem he faced with it was once again the loops. Connor was using while loop, however the code he wrote it in was ‘while’ nested loops. For some reason it did not work, however after some debugging the code was fixed. The last steps of the code were easy; all he had to do was write a portion asking the user to bring the power to the house. Notice house in the previous sentence is singular. We as a team did not add extra houses into the code, until Flatrim could get his side of the code to work with Connor’s portion. The final portion of the code, Connor implemented Greta’s code into his, so that everything was attached, and Flatrim can make the changes necessary for his portion.

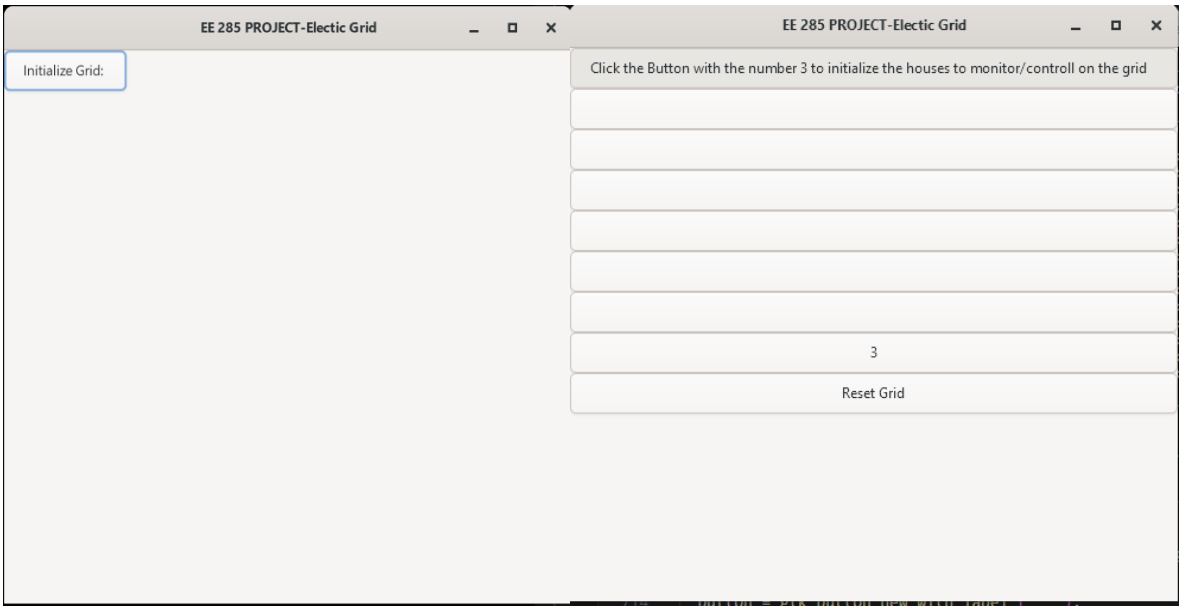
Greta used the use of math knowledge and functions to create the how much each house owes. She used one variable to be sent to the function, which was the power use of each individual house. To do this, she used a for loop to iterate through the number of houses included in the code.

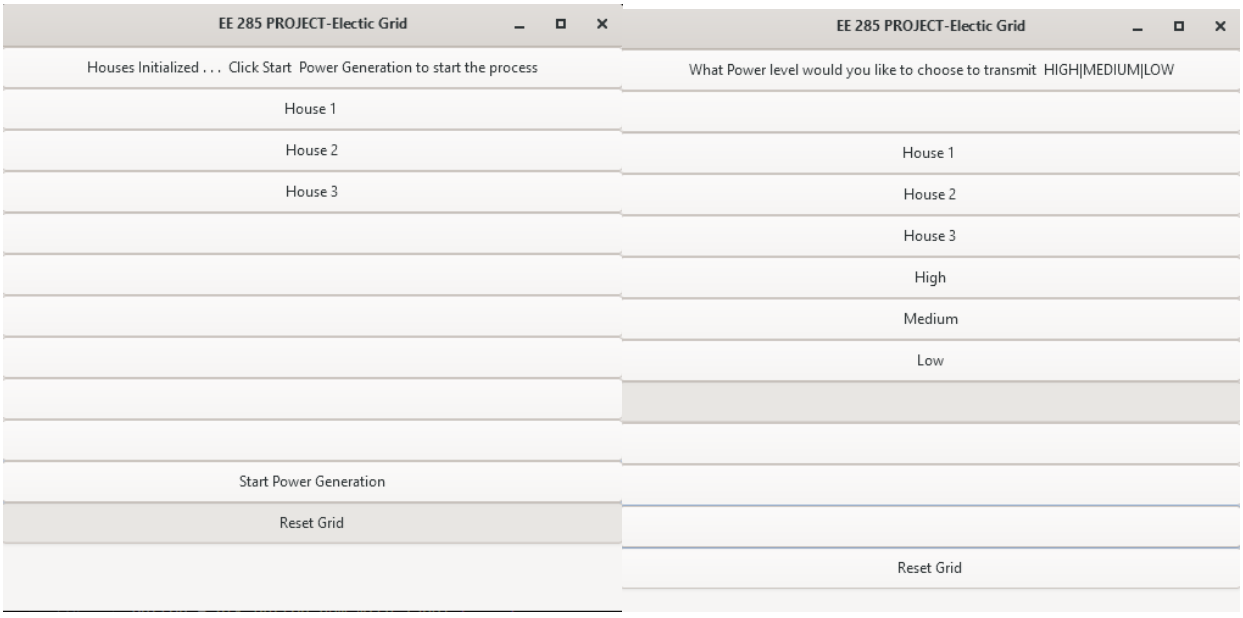
Flatrim created the user interface using the GTK library. The code implementation is a monolithic code architecture which starts at the main function at the end of the code. In the main function Flatrim decided to add a callback function when the code is activated to create a window with a

grid which is set as a child of the Window widget. In the grid Flatrim implemented a button which on callback “clicked” would call the function to initialize the grid by creating 10 buttons in which some are used as padding but most have a problem solving function, as for some reason the callback “destroy” widget was not working to remove buttons, to solve that problem we just initialize a button on top of blank buttons or on buttons which are not necessary for the decision tree. This way we can keep a consistent look without much changes to the layout besides when checking house data. After the initialize grid function and after clicking the 3 button which initializes three houses which the user can click at any time but if no power was directed to them there will not be any data to display. On this part the user can click the start power generation button which calls back to the function create_power3() which proceeds to create buttons which each callback their own function. Everything is written in button callbacks which call a function which create buttons which have callbacks and each button you choose will output you a specific button based on the decision tree you chose. Ex. From the create power function the user is prompted with 3 buttons: High, Medium, Low corresponding to power levels. On a callback of one of these buttons they call the function of their respected power level meaning if you chose high power the program calls the high3() function which initialized buttons for the user to choose which voltage line they want to use. From there if they choose the high voltage power line the program will call the HstepDown3() if anything else happens the program will call the failsafe function and the user will need to reset the program by using the Reset Grid button which calls the function to initialize the grid and resets the decision tree that was made until now. This failsafe is implemented by using the callback on click of a wrong parameter to call the reset() function. From the HstepDown3() function the user is prompted to click on a button which steps down the correct amount of voltage and in the high power case that would need to be 3 times, again at all points in the program the failsafe will be called if unsafe parameters are chosen. After choosing the correct option if the power chosen the transmission power chosen and the stepDownNumber are of HIGH value then the function send Power to house will be called when you click on a house sending the data to the dataHouse variable after which call whichever house the user chose and implement the Click for House Data button which has a callback to function final data which based on the parameters chosen in this case HIGH will create a button with the button label displaying the Wattage, Usage and Cost of the electricity that the company provides. From this point a new button will be implemented in the “Click for House Data” button where the user is allowed to start again and control other houses electricity. This decision tree will work for all correct and safe parameters that the user chooses, these include : High power, high voltage power line, 3 step downs. Medium power, medium voltage power line, 2 step downs. Low power, low voltage power line, 1 step down. The buttons are created using the : button = gtk_button_new_with_label (" label"); and then subsequently attaching that button the created grid in the beginning. The callback function is just one line: g_signal_connect (button, "clicked", G_CALLBACK (function), NULL); This line makes it so that when the button is acted upon by the clicked parameter it will call a function that it is assigned to. Using these two tools we made a decision tree style program/UI.

Results/Output

This code gives us an easy-to-use user interface that performs functions that deal with a power grid. The user can manage how much power is being produced, how many step downs there are, where the power is sent (houses), and the amount of money the household owes the company. These functions are all essential when dealing with electricity in the grid and we believe the application is extremely applicable to everyday power use. Below is a couple photos of what the User interface looks like in use.





After going through the correct parameters High->hvpl->3 step downs:



After going through unsafe parameters:



Code

```c
#include <gtk/gtk.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>


GtkWidget *window;
GtkWidget *grid;
GtkWidget *button;

int powerChosen;//3 is high power 2 is medium power 1 is low power
```

```
int transPower;//3 is for hvpl 2 for med 1 for low
int stepDownNumber;//3 is for high 2 for med 1 for low
int resetVal =0;
char dataHouse[100];
static void //reset function if user gets wrong parameters that would be
dangerous to user forces a system reset, which is inconvenient but
reset(GtkWidget *widget,
             gpointer   data)
{

    powerChosen =0;
    transPower =0;
    stepDownNumber=0;



  button = gtk_button_new_with_label ("Failsafe Has blocked that action- unsafe
parameter chosen, incident auto-reported,  Start the program over to restart");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 0, 4, 2);



  button = gtk_button_new_with_label ("  ");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 2, 4, 2);


  button = gtk_button_new_with_label ("  ");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 4, 4, 2);



  button = gtk_button_new_with_label ("  ");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 6, 4, 2);

button = gtk_button_new_with_label ("  ");
  gtk_grid_attach (GTK_GRID (grid), button, 5, 0, 10, 10);

  button = gtk_button_new_with_label ("  ");
  gtk_grid_attach (GTK_GRID (grid), button, 5, 5, 10, 10);

button = gtk_button_new_with_label ("  ");
  gtk_grid_attach (GTK_GRID (grid), button, 5, 10, 10, 10);

  button = gtk_button_new_with_label (" ");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 8, 4, 2);
```

```
  button = gtk_button_new_with_label ("  ");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 10, 4, 2);

  button = gtk_button_new_with_label ("  ");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 12, 4, 2);


  button = gtk_button_new_with_label ("  ");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 14, 4, 2);

  button = gtk_button_new_with_label ("  ");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 16, 4, 2);




}


static void //send power data to house
sendPowertoHouse(GtkWidget *widget,
             gpointer   data)
{
    if(powerChosen==3&&transPower==3&&stepDownNumber==3){


       strcpy(dataHouse,"345 kW stepped down to 13.5kW, Usage: 1214 Watts Per
Hour, Cost: 18.7 Cents per Hour");

    }else   if(powerChosen==2&&transPower==2&&stepDownNumber==2){

         strcpy(dataHouse,"345 kW stepped down to 13.5kW, Usage: 607 Watts Per
Hour, Cost: 9.390 Cents per Hour");


    }else   if(powerChosen==1&&transPower==1&&stepDownNumber==1){

        strcpy(dataHouse,"345 kW stepped down to 13.5kW, Usage: 303 Watts Per
Hour, Cost: 4.695 Cents per Hour");


    }
```

```
}



static void //callback from step down function
stepedDownPower3(GtkWidget *widget,
             gpointer   data)
{



  button = gtk_button_new_with_label ("Power was stepped down to 13.5 kW, ready
for consumer use");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 0, 4, 2);

  button = gtk_button_new_with_label ("Sending through Electrical Utility lines .
. . ");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 2, 4, 2);


  button = gtk_button_new_with_label (" Choose a house to direct the utility line
power to ");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 10, 4, 2);


button = gtk_button_new_with_label ("  ");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 12, 4, 2);

  button = gtk_button_new_with_label ("  ");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 14, 4, 2);

}




static void //Step down voltage for low power callback from low power button
LstepDown3(GtkWidget *widget,
             gpointer   data)
```

```
{
    stepDownNumber=1;
    transPower=2;

  button = gtk_button_new_with_label ("Power was transmitted to a substation you
need to step down the power for consumer use");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 0, 4, 2);

  button = gtk_button_new_with_label ("Choose step down number ");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 2, 4, 2);




  button = gtk_button_new_with_label (" 3");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 10, 4, 2);
 g_signal_connect (button, "clicked", G_CALLBACK (reset), NULL);

  button = gtk_button_new_with_label (" 2");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 12, 4, 2);
   g_signal_connect (button, "clicked", G_CALLBACK (reset), NULL);


  button = gtk_button_new_with_label (" 1");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 14, 4, 2);
  g_signal_connect (button, "clicked", G_CALLBACK (stepedDownPower3), NULL);

  button = gtk_button_new_with_label (" ");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 24, 4, 2);




}

static void //Step down voltage for med power callback from med power button
MstepDown3(GtkWidget *widget,
             gpointer   data)
{

    stepDownNumber=2;
    transPower=2;
  button = gtk_button_new_with_label ("Power was transmitted to a substation you
need to step down the power for consumer use");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 0, 4, 2);
```

```
  button = gtk_button_new_with_label ("Choose step down number ");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 2, 4, 2);




  button = gtk_button_new_with_label (" 3");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 10, 4, 2);
 g_signal_connect (button, "clicked", G_CALLBACK (reset), NULL);

  button = gtk_button_new_with_label (" 2");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 12, 4, 2);
  g_signal_connect (button, "clicked", G_CALLBACK (stepedDownPower3), NULL);


  button = gtk_button_new_with_label (" 1");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 14, 4, 2);
   g_signal_connect (button, "clicked", G_CALLBACK (reset), NULL);

  button = gtk_button_new_with_label (" ");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 24, 4, 2);




}



static void //Step down voltage for high power callback from high power button
HstepDown3(GtkWidget *widget,
             gpointer   data)
{

    stepDownNumber=3;
    transPower=3;
  button = gtk_button_new_with_label ("Power was transmitted to a substation you
need to step down the power for consumer use");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 0, 4, 2);

  button = gtk_button_new_with_label ("Choose the amount of step downs ");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 2, 4, 2);
```

```
  button = gtk_button_new_with_label (" 3");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 10, 4, 2);
g_signal_connect (button, "clicked", G_CALLBACK (stepedDownPower3), NULL);

  button = gtk_button_new_with_label (" 2");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 12, 4, 2);
   g_signal_connect (button, "clicked", G_CALLBACK (reset), NULL);


  button = gtk_button_new_with_label (" 1");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 14, 4, 2);
   g_signal_connect (button, "clicked", G_CALLBACK (reset), NULL);

  button = gtk_button_new_with_label (" ");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 24, 4, 2);




}


static void //Scenario where low  power button is clicked
low3(GtkWidget *widget,
             gpointer   data)
{
  powerChosen = 1;
  button = gtk_button_new_with_label ("You have chosen LOW power");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 0, 4, 2);

  button = gtk_button_new_with_label ("Generating 69 kW, Choose the transmission
line");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 2, 4, 2);




  button = gtk_button_new_with_label (" High Voltage Power Line");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 10, 4, 2);
  g_signal_connect (button, "clicked", G_CALLBACK (reset), NULL);
```

```
  button = gtk_button_new_with_label (" Medium Voltage Power Line ");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 12, 4, 2);
  g_signal_connect (button, "clicked", G_CALLBACK (reset), NULL);




  button = gtk_button_new_with_label (" Low Voltage Power Line");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 14, 4, 2);
  g_signal_connect (button, "clicked", G_CALLBACK (LstepDown3), NULL);

  button = gtk_button_new_with_label (" ");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 24, 4, 2);




}



static void //Scenario where med power button is clicked
med3(GtkWidget *widget,
             gpointer   data)
{
  powerChosen =2;
  button = gtk_button_new_with_label ("You have chosen MEDIUM power");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 0, 4, 2);

  button = gtk_button_new_with_label ("Generating 138.0 kW, Choose the
transmission line");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 2, 4, 2);




  button = gtk_button_new_with_label (" High Voltage Power Line");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 10, 4, 2);
  g_signal_connect (button, "clicked", G_CALLBACK (reset), NULL);
```

```
  button = gtk_button_new_with_label (" Medium Voltage Power Line ");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 12, 4, 2);
  g_signal_connect (button, "clicked", G_CALLBACK (MstepDown3), NULL);



  button = gtk_button_new_with_label (" Low Voltage Power Line");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 14, 4, 2);
  g_signal_connect (button, "clicked", G_CALLBACK (reset), NULL);



  button = gtk_button_new_with_label (" ");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 24, 4, 2);




}


static void //Scenario where high power button is clicked
high3(GtkWidget *widget,
             gpointer   data)
{
 powerChosen=3;
  button = gtk_button_new_with_label ("You have chosen HIGH power");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 0, 4, 2);

  button = gtk_button_new_with_label ("Generating 345.0 kW, Choose the
transmission line");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 2, 4, 2);




  button = gtk_button_new_with_label (" High Voltage Power Line");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 10, 4, 2);
  g_signal_connect (button, "clicked", G_CALLBACK (HstepDown3), NULL);
```

```
  button = gtk_button_new_with_label (" Medium Voltage Power Line ");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 12, 4, 2);
  g_signal_connect (button, "clicked", G_CALLBACK (reset), NULL);




  button = gtk_button_new_with_label (" Low Voltage Power Line");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 14, 4, 2);
  g_signal_connect (button, "clicked", G_CALLBACK (reset), NULL);



  button = gtk_button_new_with_label (" ");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 24, 4, 2);




}


static void //Create Power for three houses

create_Power3(GtkWidget *widget,
              gpointer   data)
{

  button = gtk_button_new_with_label (" What Power level would you like to choose
to transmit  HIGH|MEDIUM|LOW");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 0, 4, 2);

    button = gtk_button_new_with_label (" ");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 2, 4, 2);




  button = gtk_button_new_with_label (" High ");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 10, 4, 2);
  g_signal_connect (button, "clicked", G_CALLBACK (high3), NULL);
```

```
  button = gtk_button_new_with_label (" Medium ");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 12, 4, 2);
  g_signal_connect (button, "clicked", G_CALLBACK (med3), NULL);



  button = gtk_button_new_with_label (" Low ");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 14, 4, 2);
  g_signal_connect (button, "clicked", G_CALLBACK (low3), NULL);

  button = gtk_button_new_with_label (" ");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 24, 4, 2);




}




static void //H3 data based on chosen parameters
finalData3 (GtkWidget *widget,
            gpointer   data)
{
    printf("%d,%d,%d",powerChosen,transPower,stepDownNumber);

    if(powerChosen==3&&transPower==3&&stepDownNumber==3){


       button = gtk_button_new_with_label ("House 3: 345 kW stepped down to
13.5kW, Usage: 1214 Watts Per Hour, Cost: 18.7 Cents per Hour");
       gtk_grid_attach (GTK_GRID (grid), button, 5, 0, 5, 10);

        button = gtk_button_new_with_label ("Power was succesfully directed
towards House 3");
        gtk_grid_attach (GTK_GRID (grid), button, 0, 0, 4, 2);



    }else   if(powerChosen==2&&transPower==2&&stepDownNumber==2){
        button = gtk_button_new_with_label ("House 3: 345 kW stepped down to
13.5kW, Usage: 607 Watts Per Hour, Cost: 9.390 Cents per Hour");
```

```
        gtk_grid_attach (GTK_GRID (grid), button, 5, 0, 5, 10);

         button = gtk_button_new_with_label ("Power was succesfully directed
towards House 3");
        gtk_grid_attach (GTK_GRID (grid), button, 0, 0, 4, 2);



    }else   if(powerChosen==1&&transPower==1&&stepDownNumber==1){
         button = gtk_button_new_with_label ("House 3: 345 kW stepped down to
13.5kW, Usage: 303 Watts Per Hour, Cost: 4.695 Cents per Hour");
        gtk_grid_attach (GTK_GRID (grid), button, 5, 0, 5, 10);

         button = gtk_button_new_with_label ("Power was succesfully directed
towards House 3");
        gtk_grid_attach (GTK_GRID (grid), button, 0, 0, 4, 2);







    }else {
         button = gtk_button_new_with_label ("No Data Entered Yet");
        gtk_grid_attach (GTK_GRID (grid), button, 5, 0, 5, 10);}


    button = gtk_button_new_with_label ("Control/Monitor other Houses Power  ");
    gtk_grid_attach (GTK_GRID (grid), button, 5, 10, 5, 10);
    g_signal_connect (button, "clicked", G_CALLBACK (create_Power3), NULL);







}

static void //H2 data based on chosen parameters
finalData2 (GtkWidget *widget,
            gpointer   data)
{
    printf("%d,%d,%d",powerChosen,transPower,stepDownNumber);

    if(powerChosen==3&&transPower==3&&stepDownNumber==3){
```

```
       button = gtk_button_new_with_label ("House 2: 345 kW stepped down to
13.5kW, Usage: 1214 Watts Per Hour, Cost: 18.7 Cents per Hour");
       gtk_grid_attach (GTK_GRID (grid), button, 5, 0, 5, 10);

        button = gtk_button_new_with_label ("Power was succesfully directed
towards House 2");
        gtk_grid_attach (GTK_GRID (grid), button, 0, 0, 4, 2);



    }else   if(powerChosen==2&&transPower==2&&stepDownNumber==2){
        button = gtk_button_new_with_label ("House 2: 345 kW stepped down to
13.5kW, Usage: 607 Watts Per Hour, Cost: 9.390 Cents per Hour");
        gtk_grid_attach (GTK_GRID (grid), button, 5, 0, 5, 10);

         button = gtk_button_new_with_label ("Power was succesfully directed
towards House 2");
        gtk_grid_attach (GTK_GRID (grid), button, 0, 0, 4, 2);



    }else   if(powerChosen==1&&transPower==1&&stepDownNumber==1){
         button = gtk_button_new_with_label ("House 2: 345 kW stepped down to
13.5kW, Usage: 303 Watts Per Hour, Cost: 4.695 Cents per Hour");
        gtk_grid_attach (GTK_GRID (grid), button, 5, 0, 5, 10);

         button = gtk_button_new_with_label ("Power was succesfully directed
towards House 2");
        gtk_grid_attach (GTK_GRID (grid), button, 0, 0, 4, 2);




    }else {
         button = gtk_button_new_with_label ("No Data Entered Yet");
        gtk_grid_attach (GTK_GRID (grid), button, 5, 0, 5, 10);}


    button = gtk_button_new_with_label ("Control/Monitor other Houses Power  ");
    gtk_grid_attach (GTK_GRID (grid), button, 5, 10, 5, 10);
    g_signal_connect (button, "clicked", G_CALLBACK (create_Power3), NULL);
```

```
}

static void //H1 data based on chosen parameters

finalData1 (GtkWidget *widget,
            gpointer   data)
{
    printf("%d,%d,%d",powerChosen,transPower,stepDownNumber);

    if(powerChosen==3&&transPower==3&&stepDownNumber==3){


       button = gtk_button_new_with_label (" House 1: 345 kW stepped down to
13.5kW, Usage: 1214 Watts Per Hour, Cost: 18.7 Cents per Hour");
       gtk_grid_attach (GTK_GRID (grid), button, 5, 0, 5, 10);

        button = gtk_button_new_with_label ("Power was succesfully directed
towards House 1");
        gtk_grid_attach (GTK_GRID (grid), button, 0, 0, 4, 2);



    }else   if(powerChosen==2&&transPower==2&&stepDownNumber==2){
        button = gtk_button_new_with_label ("House 1: 345 kW stepped down to
13.5kW, Usage: 607 Watts Per Hour, Cost: 9.390 Cents per Hour");
        gtk_grid_attach (GTK_GRID (grid), button, 5, 0, 5, 10);

         button = gtk_button_new_with_label ("Power was succesfully directed
towards House 1");
        gtk_grid_attach (GTK_GRID (grid), button, 0, 0, 4, 2);



    }else   if(powerChosen==1&&transPower==1&&stepDownNumber==1){
         button = gtk_button_new_with_label ("345 kW stepped down to 13.5kW,
Usage: 303 Watts Per Hour, Cost: 4.695 Cents per Hour");
        gtk_grid_attach (GTK_GRID (grid), button, 5, 0, 5, 10);

         button = gtk_button_new_with_label ("Power was succesfully directed
towards House 1");
        gtk_grid_attach (GTK_GRID (grid), button, 0, 0, 4, 2);
```

```
    }else {
         button = gtk_button_new_with_label ("No Data Entered Yet");
        gtk_grid_attach (GTK_GRID (grid), button, 5, 0, 5, 10);}


    button = gtk_button_new_with_label ("Control/Monitor other Houses Power  ");
    gtk_grid_attach (GTK_GRID (grid), button, 5, 10, 5, 10);
    g_signal_connect (button, "clicked", G_CALLBACK (create_Power3), NULL);




}




static void //H1 power data button callback
houseOnePower (GtkWidget *widget,
             gpointer   data)
{


   button = gtk_button_new_with_label (" Click for House Data");
   gtk_grid_attach (GTK_GRID (grid), button, 5, 0, 5, 10);
   g_signal_connect (button, "clicked", G_CALLBACK (finalData1), NULL);


}

static void //H2 power data button callback
houseTwoPower (GtkWidget *widget,
             gpointer   data)
{


   button = gtk_button_new_with_label (" Click for House Data");
   gtk_grid_attach (GTK_GRID (grid), button, 5, 0, 5, 10);
   g_signal_connect (button, "clicked", G_CALLBACK (finalData2), NULL);
```

```
}
static void //H3 power data button callback
houseThreePower (GtkWidget *widget,
             gpointer   data)
{


   button = gtk_button_new_with_label (" Click for House Data");
   gtk_grid_attach (GTK_GRID (grid), button, 5, 0, 5, 10);
   g_signal_connect (button, "clicked", G_CALLBACK (finalData3), NULL);









}









static void //Create three houses with call back for data for each one, data
needs to be sent to a house first
threeHouseChoice (GtkWidget *widget,
             gpointer   data)
{


  button = gtk_button_new_with_label ("Houses Initialized . . .  Click Start
Power Generation to start the process");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 0, 4, 2);
```

```
   button = gtk_button_new_with_label ("House 1 ");
   gtk_grid_attach (GTK_GRID (grid), button, 0, 4, 4, 2);
   g_signal_connect (button, "clicked", G_CALLBACK (houseOnePower), NULL);




  button = gtk_button_new_with_label (" House 2 ");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 6, 4, 2);
    g_signal_connect (button, "clicked", G_CALLBACK (houseTwoPower), NULL);




  button = gtk_button_new_with_label (" House 3 ");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 8, 4, 2);
    g_signal_connect (button, "clicked", G_CALLBACK (houseThreePower), NULL);



  button = gtk_button_new_with_label ("Start Power Generation");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 24, 4, 2);
  g_signal_connect (button, "clicked", G_CALLBACK (create_Power3), NULL);


}




static void //initialize the start of the process, Creates  buttons  with
callback to start the process
initializeGridquest (GtkWidget *widget,
             gpointer   data)
{
    resetVal =0;
```

```
  button = gtk_button_new_with_label ("Click the Button with the number 3 to
initialize the houses to monitor/controll on the grid   ");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 0, 4, 2);

  button = gtk_button_new_with_label (" 3 ");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 24, 4, 2);
  g_signal_connect (button, "clicked", G_CALLBACK (threeHouseChoice), NULL);

  button = gtk_button_new_with_label ("  ");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 10, 4, 2);

  button = gtk_button_new_with_label ("  ");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 12, 4, 2);


  button = gtk_button_new_with_label ("  ");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 14, 4, 2);

  button = gtk_button_new_with_label ("  ");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 16, 4, 2);

  button = gtk_button_new_with_label ("  ");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 18, 4, 2);

  button = gtk_button_new_with_label ("  ");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 20, 4, 2);

  button = gtk_button_new_with_label ("Reset Grid");
  gtk_grid_attach (GTK_GRID (grid), button, 0, 26, 4, 2);
  g_signal_connect (button, "clicked", G_CALLBACK (initializeGridquest), NULL);


}



static void //Start the application by creating a 500x500 pixel window
activate (GtkApplication *app,
          gpointer        user_data)
{


  //Window widget
  window = gtk_application_window_new (app);
```

```
  gtk_window_set_title (GTK_WINDOW (window), "EE 285 PROJECT-Electic Grid");
  gtk_window_set_default_size (GTK_WINDOW (window), 500, 500);

  grid = gtk_grid_new ();
  gtk_window_set_child (GTK_WINDOW (window), grid);




  button = gtk_button_new_with_label ("Initialize Grid: ");
  g_signal_connect (button, "clicked", G_CALLBACK (initializeGridquest), NULL);
  gtk_grid_attach (GTK_GRID (grid), button, 1, 0, 1, 1);




  //Window
  gtk_window_present (GTK_WINDOW (window));
}


//main done according to GTK doc
int
main (int    argc,char **argv)
{
  GtkApplication *app;
  int status;

  app = gtk_application_new ("ee285.project", G_APPLICATION_DEFAULT_FLAGS);
  g_signal_connect (app, "activate", G_CALLBACK (activate), NULL);//activate
window when code compiles
  status = g_application_run (G_APPLICATION (app), argc, argv);
  g_object_unref (app);// essentially free() function; decreases the app
reference count, when reference is 0 object is finalized(memory is freed) in our
case app object



    if (resetVal ==1){

//Grid widget where buttons are placed
  button = gtk_button_new_with_label (" Restart Grid");
  gtk_grid_attach (GTK_GRID (grid), button, 20, 0, 4, 2);
```

```
  g_signal_connect (button, "clicked", G_CALLBACK (initializeGridquest), NULL);
    }
  return status;
}
```

References

"Electricity Rates by State: April 2024: Chooseenergy.Com®." *April 2024 | ChooseEnergy.Com®*, www.chooseenergy.com/electricity-rates-by-state/. Accessed 9 Apr. 2024.